Constructors in Java

In Java, a constructor is a block of codes similar to the method. It is called when an instance of the class is created. At the time of calling constructor, memory for the object is allocated in the memory.

It is a special type of method which is used to initialize the object.

Every time an object is created using the new() keyword, at least one constructor is called.

It calls a default constructor if there is no constructor available in the class. In such case, Java compiler provides a default constructor by default.

There are two types of constructors in Java: no-arg constructor, and parameterized constructor.

Note: It is called constructor because it constructs the values at the time of object creation. It is not necessary to write a constructor for a class. It is because java compiler creates a default constructor if your class doesn't have any.

Rules for creating Java constructor

There are two rules defined for the constructor.

1. Constructor name must be the same as its class name
2. A Constructor must have no explicit return type
3. A Java constructor cannot be abstract, static, final, and synchronized

Note: We can use access modifiers while declaring a constructor. It controls the object creation. In other words, we can have private, protected, public or default constructor in Java.

Types of Java constructors

There are two types of constructors in Java:

1. Default constructor (no-arg constructor)
2. Parameterized constructor

Java Default Constructor

A constructor is called "Default Constructor" when it doesn't have any parameter. **Syntax of default constructor:**

<class_name>(){}

Example of default constructor

In this example, we are creating the no-arg constructor in the Bike class. It will be invoked at the time of object creation.

```
//Java Program to create and call a default constructor
class Bike1
{
        Bike1()
        {
                System.out.println("Bike is created");
        }
        public static void main(String args[])
        {
                Bike1 b=new Bike1();
        }
}
```

Example of default constructor that displays the default values

```
class Student3
{
        int id;
        String name;

        void display()
        {
                System.out.println(id+" "+name);
        }

        public static void main(String args[])
        {

                Student3 s1=new Student3();
                Student3 s2=new Student3();

                s1.display();
                s2.display();
        }
}
```

Explanation the above class, you are not creating any constructor so compiler provides you a default constructor. Here 0 and null values are provided by default constructor.

Java Parameterized Constructor

A constructor which has a specific number of parameters is called a parameterized constructor.

Why use the parameterized constructor?

The parameterized constructor is used to provide different values to distinct objects. However, you can provide the same values also.

Example of parameterized constructor

In this example, we have created the constructor of Student class that have two parameters. We can have any number of parameters in the constructor.

```
class Student4
{
        int id;
        String name;

        Student4(int i,String n)
        {
        id = i;
        name = n;
        }
        void display()
        {
                System.out.println(id+" "+name);
        }

        public static void main(String args[])
        {
        Student4 s1 = new Student4(111,"Karan");
        Student4 s2 = new Student4(222,"Aryan");

                s1.display();
                s2.display();
        }
}
```

Constructor Overloading in Java

In Java, a constructor is just like a method but without return type. It can also be overloaded like Java methods.

Constructor overloading in Java is a technique of having more than one constructor with different parameter lists. They are arranged in a way that each constructor performs a different task. They are differentiated by the compiler by the number of parameters in the list and their types.

Example of Constructor Overloading

```
class Student5
{
      int id;
      String name;
      int age;
      //creating two arg constructor
      Student5(int i,String n)
      {
          id = i;
          name = n;
      }
      //creating three arg constructor
      Student5(int i,String n,int a)
      {
          id = i;
          name = n;
          age=a;
      }
      void display()
      {
      System.out.println(id+" "+name+" "+age);  }

      public static void main(String args[])
      {
          Student5 s1 = new Student5(111,"Karan");
         Student5 s2 = new Student5(222,"Aryan",25);
         s1.display();
          s2.display();
      }
}
```

Java Copy Constructor

There is no copy constructor in Java. However, we can copy the values from one object to another like copy constructor in C++.

There are many ways to copy the values of one object into another in Java. They are:

- By constructor
- By assigning the values of one object into another
- By clone() method of Object class

In this example, we are going to copy the values of one object into another using Java constructor.

```
class Student6
{
    int id;
    String name;
    Student6(int i,String n)
    {
        id = i;
        name = n;
    }
    Student6(Student6 s)
    {
        id = s.id;
        name =s.name;
    }
    void display()
    {
    System.out.println(id+" "+name);
    }

    public static void main(String args[])
    {
        Student6 s1 = new Student6(111,"Karan");
        Student6 s2 = new Student6(s1);
        s1.display();
        s2.display();
    }
}
```

Copying values without constructor

We can copy the values of one object into another by assigning the objects values to another object. In this case, there is no need to create the constructor.

```
class Student7
{
    int id;
    String name;
```

```
    Student7(int i,String n)
    {
        id = i;
        name = n;
    }
    Student7()
    {}
    void display()
    {
    System.out.println(id+" "+name);
    }

    public static void main(String args[])
    {
        Student7 s1 = new Student7(111,"Karan");
        Student7 s2 = new Student7();

            s2.id=s1.id;
        s2.name=s1.name;

            s1.display();
        s2.display();
}
}
```